

Beyond Time Complexity: Data Movement Complexity Analysis for Matrix Multiplication

Wesley Smith

Aidan Goldfarb

Chen Ding

wsmith6@cs.rochester.edu

agoldfa7@u.rochester.edu

cding@cs.rochester.edu

University of Rochester

Rochester, NY, USA

Abstract

Data movement is becoming the dominant contributor to the time and energy costs of computation across a wide range of application domains. However, time complexity is inadequate to analyze data movement. This work expands upon Data Movement Distance, a recently proposed framework for memory-aware algorithm analysis, by 1) demonstrating that its assumptions conform with microarchitectural trends, 2) applying it to four variants of matrix multiplication, and 3) showing it to be capable of asymptotically differentiating algorithms with the same time complexity but different memory behavior, as well as locality optimized vs. non-optimized versions of the same algorithm. In doing so, we attempt to bridge theory and practice by combining the operation count analysis used by asymptotic time complexity with per-operation data movement cost resulting from hierarchical memory structure. Additionally, this paper derives the first fully precise, fully analytical form of recursive matrix multiplication's miss ratio curve on LRU caching systems. Our results indicate that the Data Movement Distance framework is a powerful tool going forward for engineers and algorithm designers to understand the algorithmic implications of hierarchical memory.

CCS Concepts: • Theory of computation → Design and analysis of algorithms.

Keywords: matrix multiplication, hierarchical memory, algorithm analysis, data movement

ACM Reference Format:

Wesley Smith, Aidan Goldfarb, and Chen Ding. 2022. Beyond Time Complexity: Data Movement Complexity Analysis for Matrix Multiplication. In *2022 International Conference on Supercomputing (ICS '22)*, June 28–30, 2022, Virtual Event, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3524059.3532395>

1 INTRODUCTION

In exascale computing, the cost of data movement exceeds that of computation [6]: as such, data movement is a key factor in not only system performance but also in the computer science community's growing responsibility to address computing's role in the climate crisis.

Optimizing a program or a system for locality is difficult, as modern memory systems are large and complex. For portability, we should not program data movement directly, but we should be aware of its cost. However, there does not yet exist a single quantity that can characterize the effect of locality optimization at the program or algorithm level.

Standard techniques for understanding algorithm-hierarchy interactions, like miss ratio analysis, yield insight for algorithm designers with a target set of machine parameters in mind, but do not allow for general understanding of an algorithm's intrinsic data movement for an arbitrary target machine.

The actual effect of cache usage depends on all components of a program and also its running environment. Assumptions about a machine may be wrong, imprecise, or soon obsolete. A program may run on a remote computer in a public or commercial computing center with limited information about its memory system. Auto-tuning can select the best parameters for a given system, but it is difficult to tune if a system is shared.

In a recent position paper, Ding and Smith [7] defined an abstract measure of memory cost called Data Movement Distance (DMD). Memory complexity is measured by DMD in the same way time complexity is by operation count. They showed results for two types of data traversals with only constant factor differences in DMD.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICS '22, June 28–30, 2022, Virtual Event, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9281-5/22/06...\$15.00

<https://doi.org/10.1145/3524059.3532395>

This paper presents DMD analysis for different approaches to matrix multiplication. It differs from past work in several ways. First, unlike practical analysis, i.e. those based on miss ratios, DMD analysis is asymptotic and machine agnostic. Second, unlike I/O complexity, DMD analysis includes the effect of a cache hierarchy. In addition, the derivation is radically different from past solutions. For example, efficient algorithms often make use of temporaries that are dynamically allocated. They share cache, but the cache sharing is not analyzed by past complexity analysis. It is measured by practical analysis in concrete terms, i.e. cache misses, not asymptotic terms.

DMD measures data movement. Running time depends also on latency tolerance techniques, especially prefetching, which is outside the scope of this paper. Latency tolerance techniques, however, do not reduce the amount of data movement intrinsic to the algorithm. In addition, this paper targets sequential computation and assumes a memory hierarchy that is concentric, layered and managed using the least-recently-used (LRU) cache replacement policy.

2 MAIN CONTRIBUTIONS

The focus of this work is exploring the algorithmic implications of hierarchical memory systems by fleshing out the memory-aware algorithm analysis framework Data Movement Distance (DMD) introduced in [7]. The main contributions are as follows:

- Derivation and empirical validation of the first fully precise analytical form of recursive matrix multiplication's miss ratio on LRU caching systems,
- Application of the DMD framework for memory-aware algorithm analysis to four variants of matrix multiplication,
- Exploration of the effects of locality optimizations on the previous results,

In addition, we expand the motivation for and justification of the DMD framework by demonstrating the following:

- microarchitectural trends in cache memory conform with the framework's assumptions,
- DMD is capable of asymptotically differentiating algorithms with the same time complexity as a result of their memory behavior, as well as asymptotically differentiating between locality optimizations

Taken together, we believe that the results of our analyses indicate that the DMD framework is a powerful tool going forward for engineers and algorithm designers to understand the algorithmic implications of hierarchical memory.

3 BACKGROUND AND MOTIVATION

3.1 Locality Concepts

The locality concept most central to this work is *reuse distance* (RD) [27]. Reuse distance, or LRU stack distance [14],

characterizes an individual memory access by counting the number of distinct memory locations accessed by the program between the most recent previous use of that memory location and the current use.

For example, let letters denote distinct memory locations in the following access trace:

abbca

In this example, the reuse distance of the second access to *b* is 1, as only *b* occurs in the window from position 2 to position 3, while the reuse distance of the second access to *a* is 3, as *a, b, c* all occur in the window from position 1 to position 5.

Reuse distance and miss ratio for fully-associative LRU cache are interconvertible, with their relation as follows:

$$MR(c) = P(rd > c)$$

Accesses with reuse distance greater than *c* are misses in LRU caches of size *c* or less. So, reuse distance distributions and miss ratio curves are the same information.

3.2 Data Movement Distance

The most ubiquitous technique for algorithm cost analysis is asymptotic time complexity, which measures operation count as a function of input size. However, on machines with hierarchical memory, execution cost will scale with input size *faster* than time complexity would indicate, because as data size increases, more program data must be stored in large, slow hierarchy components: the cost of data movement scales with input size as well. Data Movement Distance (DMD) is a novel framework for memory-aware algorithm analysis proposed by first Snyder and Ding [20] and then Ding and Smith [7] in which operation count is combined with per-access data movement cost by considering the algorithm's reuse distance distribution.

Because data movement cost varies across machines, the DMD framework includes an abstracted version of a memory hierarchy, termed the *geometric stack*, on which the behavior of algorithms is considered. The geometric stack can be understood to be an infinite-level memory hierarchy in which each level stores a single datum. The cost of accessing the datum at level *n* is $\sqrt{n}$. DMD for a program is then defined as follows:

Definition 1 (Data Movement Distance). *For a program p with data accesses a_i , let the reuse distance of a_i be d_i . The DMD for p under caching algorithm A is*

$$DMD(p) = \sum_i \sqrt{d_i}$$

In words, a program's DMD is the sum of the square roots of its memory accesses' reuse distances. As such, we arrive at our first theorem:

Theorem 1 (DMD bounds). *Let the time complexity of program p be $O(f(n))$ and let its space complexity be $O(g(n))$. Then*

$$f(n) \leq \text{DMD}(p) \leq f(n) \cdot \sqrt{g(n)}$$

where $\text{DMD}(p)$ is asymptotic (big- O) DMD.

Proof. It suffices to note that this program will have $f(n)$ memory accesses, the minimum value a reuse distance can take is 1, and the maximum value a reuse distance can take is $g(n)$ (data size). Then, Definition 1 yields the above. $\square$

Memory complexity is measured by DMD in the same way time complexity is by operation count. As Theorem 1 shows, DMD is at least time complexity, because every operation accesses some data. In the following analysis of matrix multiplication, DMD is asymptotically greater than time complexity, and we specifies constant co-efficients. Indeed, DMD is useful only when it is greater than time complexity; otherwise constant-size memory is sufficient, and the memory problem is trivial, i.e. *compute bound*.

An intuitive explanation for the $\sqrt{n}$ cost function is that it is the distance the data must travel if we represent the infinite-level hierarchy as a series of concentric 2-D shapes (such as circles) and let area represent capacity. In the following section, we will discuss the microarchitectural trend that this cost function reflects. In Ding and Smith’s formulation for DMD, cache replacement policy is a parameter, however in the derivations in this paper we will use LRU replacement.

Throughout this paper we will use the asymptotic equivalence notation $\sim ()$, which is identical to big- O notation except it retains primary factor coefficients. DMD measures the memory cost and is *complexity without Big- O* .

3.3 Why $\sqrt{n}$?

At the core of the DMD framework is the notion that we need a relationship between stack position, or reuse distance, and cost. Ding and Smith [7] use $\sqrt{n}$ with the argument that it reflects physical memory layout. We expand on that here by demonstrating that $\sqrt{n}$ also reflects the cost of memory access on modern architectures. Figure 1 demonstrates access latency as a function of distance from the processor for cache sizes and latencies corresponding to the AMD Zen2 and Intel Sunny Cove architectures (numbers from [5]) up to $\approx 2\text{MB}$ data size. As stack position increases, data is forced into slower caches and its access latency increases. While “distance from the processor” is not a perfect match with LRU stack distance on optimized hardware and Figure 1 contains no empirical measurements, it is clear that the general trend in these architectures’ latencies is well captured by the $\sqrt{x}$ family of functions.

This relationship has been observed in other contexts as well: Yavitz et al. [26] show that access latency scales with the square root of cache size, and Cassidy and Andreou [4] demonstrate that energy cost behaves similarly. These

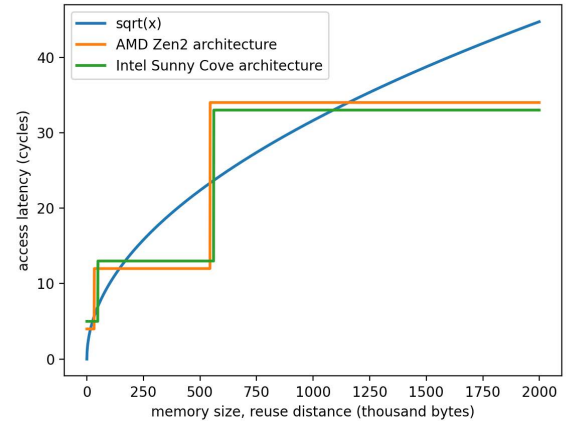


Figure 1. Access latencies of modern microarchitectures as a function of stack position

results, and thus the square root concept, have been used in the design of cutting edge memory systems and techniques, such as Tsai et al.’s Jenga [23].

3.4 Why Measure Hierarchical Locality?

DMD is to measure hierarchical locality. A program has *hierarchical locality* if it makes use of a *cache hierarchy*, where there is has more than a single level of cache, and the cache size and organization may vary from machine to machine. Such cache hierarchies are the norm on today’s machines.

As a contrast, consider single-cache locality which means programming to utilize a cache of a specific size. Single-cache locality is unreliable for two reasons. The first is portability. The actual cache usage depends on the choice of programming languages, compilers, and target machines. The second problem is environmental, e.g. interference from run-time systems and from peer programs that share the same cache. Single-cache locality is not a robust program property because of these two sources of uncertainty.

Hierarchical locality is portable and elastic. It is independent of implementation, and it runs well in a shared environment. Cache oblivious algorithms, e.g. recursive matrix multiplication, were developed for hierarchical locality. Next, we use DMD to analyze this effect.

4 RECURSIVE MATRIX MULTIPLICATION MISS RATIO ANALYSIS

In this section we derive what is to our knowledge the first fully precise analytical form of recursive matrix multiplication’s (RMM) miss ratio curve on LRU memory. Previously, RMM’s asymptotic cache behavior has been derived [18], but we derive the precise, numeric miss ratio for all cache sizes and matrix sizes and validate our model’s accuracy against instrumented executions.

We will derive miss ratio by deriving the distribution of reuse distances incurred by RMM. Reuse distance and miss ratio have been shown to be interconvertible [27], meaning they are the same information.

At a high level, our approach will be as follows:

1. decompose the recursive algorithm into its canonical tree structure,
2. split memory accesses into accesses to temporaries and accesses to input matrices,
3. derive symbolic representations of each type of memory access pattern at each level of the tree,
4. iterate over the entire tree to create the full reuse distribution.

The full derivation is too lengthy to include in this paper; the interested reader may find a more detailed version at github.com/wes-smith/Tech-Report-ICS-22/. We will present the lemmas and theorems that constitute the majority of the contribution of this section as well as the skeleton of the full derivation, but will elide in-depth proofs of many of the derived relationships. We demonstrate correctness by implementing our model and showing its functional equivalence to an instrumented version of RMM.

```

Function rmm(A,B):
  n = A.rows
  let C be a new nxn matrix
  if n == 1:
    C11 = A11 * B11
  else:
    C11 = rmm(A11, B11) + rmm(A12, B21)
    C12 = rmm(A11, B12) + rmm(A12, B22)
    C21 = rmm(A21, B11) + rmm(A22, B12)
    C22 = rmm(A21, B12) + rmm(A22, B22)
  return C

```

The above contains the pseudocode for standard recursive matrix multiplication, and is the form for RMM that we will analyze. Note that this is RMM at its most basic: we don't consider optimizations such as using a base case larger than 1×1 or temporary reclamation and reuse. We will consider such optimizations when analyzing the algorithm's data movement distance, but for our miss ratio/reuse distance derivation we analyze RMM in its simplest form. Here, each call to multiply $N \times N$ matrices decomposes into eight recursive calls, each multiplying $\frac{N}{2} \times \frac{N}{2}$ matrices. After each pair of recursive calls, there is an addition step.

Figure 2 contains the decomposition of a 4×4 multiplication. The blue node, marked L4, represents a 4×4 multiplication, the orange nodes, marked L2, represent 2×2 multiplications, and the green nodes, marked L1, represent 1×1 multiplications. Execution is a depth-first, left-to-right traversal of this tree. In the rest of this derivation, we will use the notation LN to refer to a node in the tree that is multiplying $N \times N$ matrices.

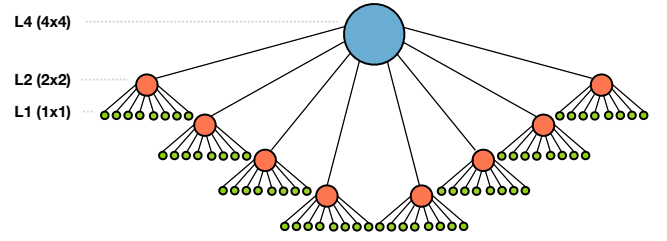


Figure 2. The tree structure of recursive matrix multiplication

4.1 Temporaries

We first analyze the behavior of RMM's temporary usage:

Definition 2 (Temporary Count). *Let T_N represent the number of temporaries introduced in an LN call to recursive matrix multiplication, i.e. multiplying $N \times N$ matrices. Then*

$$T_N = \sum_{i=0}^{\log_2(N)} 8^{\log_2(N)-i} \cdot (2^i)^2 = N^2 \cdot (2N - 1).$$

Having a uniform branching factor makes deriving the node count at each level trivial:

Definition 3. *In an $N \times N$ recursive matrix multiplication, the number of LX nodes is as follows:*

$$\#LX = 8^{\log_2(N) - \log_2(X)} = \frac{N^3}{X^3}$$

The next step is to exploit the tree's symmetry to understand how much repetition there is in the reuse distances of temporaries:

Lemma 1 (Temporary Symmetry). *Let $F_T(i, j, N, a)$ denote the reuse distance of the (i, j) -th element of the a -th temporary matrix introduced at LN . Then*

$$F_T(i, j, N, a) = F_T(i, j, N, (a \% 2))$$

Lemma 1 has a straightforward high-level interpretation: at LN , there are only at most two $N \times N$ matrices' worth of temporaries with different reuse distances. These correspond to the first and second elements in one of the additions in the RMM pseudocode at the beginning of Section 4, with each addition group introducing temporaries with identical behavior.

We will denote these two temporary matrices $DT_{1,N}$ and $DT_{2,N}$. We elide their derivations, but the approach involved a mix of purely analytical analysis of RMM's tree structure and pattern extraction from empirical results from an instrumented version of RMM. Their forms are as follows, where ellipses indicate the same value in all columns or values decreasing by 1 per column:

Lemma 2 (First Temporary Matrix). *Let $DT_{1,N}$ denote the matrix of reuse distances of temporaries introduced at level N*

in the first node of an addition group. Then

$$DT_{1,N} = \begin{bmatrix} d_1 & \dots & d_1 - (\frac{n}{2} - 1) & d_2 - (\frac{n}{2})^2 + \frac{n}{2} & \dots & d_2 - (\frac{n}{2})^2 + 1 \\ d_1 & \dots & d_1 - (\frac{n}{2} - 1) & d_2 - (\frac{n}{2})^2 + n & \dots & d_2 - (\frac{n}{2})^2 + \frac{n}{2} + 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ d_1 & \dots & d_1 - (\frac{n}{2} - 1) & d_2 & \dots & d_2 - \frac{n}{2} - 1 \\ d_1 - \phi(n) & \dots & d_1 - \phi(n) - (\frac{n}{2} - 1) & d_2 - \delta(n) - (\frac{n}{2})^2 + \frac{n}{2} & \dots & d_2 - \delta(n) - (\frac{n}{2})^2 + 1 \\ d_1 - \phi(n) & \dots & d_1 - \phi(n) - (\frac{n}{2} - 1) & d_2 - \delta(n) - (\frac{n}{2})^2 + n & \dots & d_2 - \delta(n) - (\frac{n}{2})^2 + \frac{n}{2} + 1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ d_1 - \phi(n) & \dots & d_1 - \phi(n) - (\frac{n}{2} - 1) & d_2 - \delta(n) & \dots & d_2 - \delta(n) - \frac{n}{2} - 1 \end{bmatrix}$$

where

$$\delta(N) = N^3$$

$$\phi(N) = N^3 - \frac{N^2}{2}$$

$$d_1 = \lfloor 2DN - (2 \cdot (T_{\frac{N}{2}} - 2((\frac{N}{2})^2 - 1))) \rfloor$$

$$d_2 = \lfloor 2DN - (4T_{\frac{N}{2}} + 2((\frac{N}{2})^2 - 2(\frac{N}{2} - 1) - (2(\frac{N}{2})^2 - \frac{N}{2}))) \rfloor$$

Lemma 3 (Second Temporary Matrix). *Let $DT_{2,N}$ denote the matrix containing the reuse distances of the temporaries introduced at level N in the second node of an addition group. Then*

$$DT_{2,N} = \begin{bmatrix} d_3 & \dots & d_3 & d_4 & \dots & d_4 \\ d_3 + n & \dots & d_3 + n & d_4 + \frac{3N}{2} & \dots & d_4 + \frac{3N}{2} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ d_3 + n(\frac{n}{2} - 1) & \dots & d_3 + n(\frac{n}{2} - 1) & d_4 + (\frac{n}{2} - 1)(\frac{3N}{2}) & \dots & d_4 + (\frac{n}{2} - 1)(\frac{3N}{2}) \\ d_3 - \gamma(N) & \dots & d_3 - \gamma(N) & d_4 - \omega(N) & \dots & d_4 - \omega(N) \\ d_3 - \gamma(N) + n & \dots & d_3 - \gamma(N) + n & d_4 - \omega(N) + \frac{3N}{2} & \dots & d_4 - \omega(N) + \frac{3N}{2} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ d_3 - \gamma(N) + n(\frac{n}{2} - 1) & \dots & d_3 - \gamma(N) + n(\frac{n}{2} - 1) & d_4 - \omega(N) + (\frac{n}{2} - 1)(\frac{3N}{2}) & \dots & d_4 - \omega(N) + (\frac{n}{2} - 1)(\frac{3N}{2}) \end{bmatrix}$$

where

$$\gamma(N) = N^3 - N^2$$

$$\omega(N) = N^3 - \frac{N^2}{2}$$

$$d_3 = \lfloor DN - (2 \cdot T_{\frac{N}{2}} - (2(\frac{N}{2})^2 - 1)) \rfloor$$

$$d_4 = \lfloor DN - (2(\frac{N}{2})^2) - (4T_{\frac{N}{2}} - 2(\frac{N}{2})^2 + 2) - ((\frac{N}{2})^2 - N) + (\frac{N}{2} + 1) \rfloor$$

The previous two lemmas contain a large amount of information and can be hard to parse: the takeaway should be that we can symbolically characterize the reuse distances of temporaries in terms of where in the tree they are created and used. The interested reader can see github.com/wes-smith/Tech-Report-ICS-22/ for more information on how the above functions and relationships were derived, but the exact specifics are not essential to understand the larger picture.

Lemmas 2 and 3, which consider the reuse distances incurred by temporaries in individual nodes of our tree, will be later combined with tree structure information to create the complete picture of temporary reuse.

4.2 Matrices A and B

To understand the behavior of the data in matrices A and B, we must first define what it means for a reuse of one such

datum to be at LN given that all the accesses to A and B occur in leaves of the tree.

Definition 4 (Input data reuse levels). *When referring to a reuse of data in matrix A or B, LN indicates the **largest** N for which there exists a complete LN call between the data item's use and reuse.*

To help visualize, Figure 3 contains the tree decomposition of a 4x4 matrix multiplication (L4 call) with the nodes that contain a reference to element A[1, 1] (1-indexed: the top-left-most element of matrix A) highlighted. There are two L1 reuses, between the two blue nodes and between the two pink nodes, and one L2 reuse, between the second blue node and the first pink node (as there exists a full L2 call between those two). The dashed line boxes indicated addition groups. Combining with tree structure, we can count occurrences of

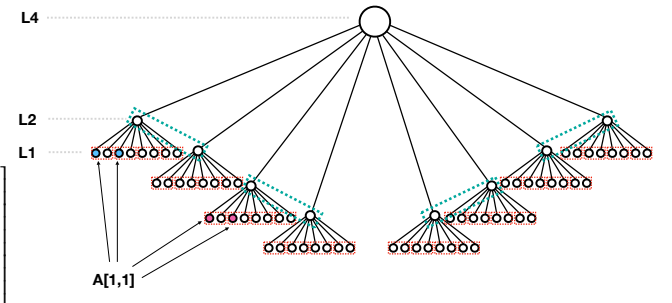


Figure 3. Tree decomposition with A[1,1] accesses highlighted

LN reuse for all N:

Lemma 4. *Let $RC(N, M)$ denote the number of LM reuses of any item in matrix A or B in an LN call (an $N \times N$ multiplication). Then*

$$RC(N, M) = \frac{N}{2M}$$

With an understanding of the frequency of each type of reuse of data from A and B we can turn our focus to computing the values of the reuse distances. Our high level angle of attack for this will be to again decompose into temporaries vs. input data: we will derive separately the number of temporaries as well as the number of elements of A and B between two consecutive uses of an item from A or B. Formally:

Definition 5 (Reuse distance decomposition). *Let $F(i, j, N)$ represent the reuse distance of element A[i, j] at an LN reuse. Then*

$$F(i, j, N) = f_T(i, j, N) + f_{A,B}(i, ((j - 1) \% N) + 1, N),$$

where $f_T(i, j, N)$ is the number of unique temporaries within the A[i, j] LN reuse pair, and $f_{A,B}(i, j, N)$ is the number of

unique elements of matrices A, B within the same reuse pair. Additionally,

$$G(i, j, N) = g_T(i, ((j-1) \% N) + 1, N) + g_{A,B}(i, j, N),$$

for matrix B .

We will now derive the four summed terms in the Definition 5 (two in F , two in G).

4.2.1 Matrix A. We again elide the full derivations of f_T and $f_{A,B}$ but point the interested reader to github.com/wes-smith/Tech-Report-ICS-22/. The results are as follows:

Lemma 5 (Matrix A temporaries). *Let $f_T(i, j, N)$ be the number of unique temporaries within an $A[i, j]$ LN reuse pair. Then*

$$\begin{aligned} f_T(i, j, N) &= T_N + 2N^2 + 8T_{\frac{N}{2}} - \sum_{i=0}^{\log_2(N)-1} 2T_{2^k} \\ &+ \sum_{k=0}^{\log_2(N)+1} (2^k)^2 \cdot I(((j-1) \% 2N) \% 2^{k+1}) + 1 > 2^k) \\ &+ \sum_{k=0}^{\log_2(N)} (2^k)^2 \cdot I(((i-1) \% N) \% 2^{k+1}) + 1 > 2^k) \end{aligned}$$

For $f_{A,B}$, we isolate the following recursive relationship (where the recursion is between parents and children in the tree) as being essential:

$$\begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix} \rightarrow \begin{bmatrix} \delta_1 & \delta_1 + N^2 \\ N^2 & 2 \cdot N^2 \\ 2 \cdot N^2 & N^2 \\ \delta_2 + N^2 & \delta_2 \end{bmatrix}$$

Here, each matrix entry is itself a $\frac{N}{2} \times N$ matrix. Lemma 6 reflects this recursion, with the eight terms corresponding to the eight sections of the second matrix. The indicator function calls isolate which section of the matrix (i, j) fall in, and the four recursive calls correspond to the four sections that contain a δ term.

Lemma 6. *Let $f_{A,B}(i, j, N)$ be the number of unique items in matrices A and B within an $A[i, j]$ LN reuse pair.*

$$f_{A,B}(i, j, N) = 4N^2 + f'_{A,B}(i, j, N)$$

where

$$\begin{aligned} f'_{A,B}(i, j, N) &= 4 \cdot N^2 + \left(\frac{N}{2}\right)^2 \cdot I\left(\frac{N}{4} < i \leq \frac{N}{2}, j \leq \frac{N}{2}\right) + 2\left(\frac{N}{2}\right)^2 \cdot I\left(\frac{N}{4} < i \leq \frac{N}{2}, \frac{N}{2} < j \leq N\right) \\ &+ \left(\frac{N}{2}\right)^2 \cdot I\left(\frac{N}{2} < i \leq \frac{3N}{4}, \frac{N}{2} < j \leq N\right) + 2\left(\frac{N}{2}\right)^2 \cdot I\left(\frac{N}{2} < i \leq \frac{3N}{4}, j \leq \frac{N}{2}\right) \\ &+ I\left(i > \frac{3N}{4}, j \leq \frac{N}{2}\right) \cdot \left(\left(\frac{N}{2}\right)^2 + f_{A,B}\left(i - \frac{N}{2}, j, \frac{N}{2}\right)\right) + I\left(i > \frac{3N}{4}, j > \frac{N}{2}\right) \cdot \left(f_{A,B}\left(i - \frac{N}{2}, j - \frac{N}{2}, \frac{N}{2}\right)\right) \\ &+ I\left(i \leq \frac{N}{4}, j > \frac{N}{2}\right) \cdot \left(\left(\frac{N}{2}\right)^2 + f_{A,B}\left(i, j - \frac{N}{2}, \frac{N}{2}\right)\right) \\ &+ I\left(i \leq \frac{N}{4}, j \leq \frac{N}{2}\right) \cdot \left(f_{A,B}\left(i, j, \frac{N}{2}\right)\right) \end{aligned}$$

and

$$f'_{A,B}(i, j, 2) = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix}$$

4.2.2 Matrix B. The quantities of interest for matrix B have very similar forms:

Lemma 7 (Matrix B temporaries). *Let $g_T(i, j, N)$ be the number of unique temporaries within an $B[i, j]$ LN reuse pair. Then*

$$\begin{aligned} g_T(i, j, N) &= 4 \cdot T_N + 2N^2 - \sum_{k=0}^{\log_2(N)-1} 4 \cdot T_{2^k} + \\ &\sum_{k=0}^{\lceil \log_2(N) \rceil} 4^k \cdot I(((i-1) \% 2^{k+1}) + 1 > 2^k) \\ &+ \sum_{k=0}^{\lceil \log_2(N) \rceil} 4^k \cdot I(((j-1) \% 2^{k+1}) + 1 > 2^k) \end{aligned}$$

Again, we extract the key recursive relationship between reuse distances in parents and children in the tree:

$$\begin{bmatrix} \delta_1 & \delta_2 \end{bmatrix} \rightarrow \begin{bmatrix} \delta_1 & 2N^2 & 6N^2 & \delta_2 + (2N)^2 \\ \delta_1 + (2N)^2 & 6N^2 & 2N^2 & \delta_2 \end{bmatrix}$$

where each entry represents an $N \times \frac{N}{2}$ matrix. Lemma 8 again reflects this recursion:

Lemma 8. *Let $g_{A,B}(i, j, N)$ be the number of unique items in matrices A and B within an $B[i, j]$ LN reuse pair. Then*

$$g_{A,B}(i, j, N) = 6N^2 + g'_{A,B}(i, j, N)$$

where

$$\begin{aligned} g'_{A,B}(i, j, N) &= \frac{N^2}{2} \cdot I\left(i \leq N, \frac{N}{2} < j \leq N\right) + \left(\frac{3N^2}{2}\right) \cdot I\left(i \leq N, N < j \leq \frac{3N}{2}\right) \\ &+ \left(\frac{3N^2}{2}\right) \cdot I\left(N < i \leq 2N, \frac{N}{2} < j \leq N\right) + \left(\frac{N^2}{2}\right) \cdot I\left(N < i \leq 2N, N < j \leq \frac{3N}{2}\right) \\ &+ I\left(i \leq N, j \leq \frac{N}{2}\right) \cdot g'_{A,B}\left(i, j, \frac{N}{2}\right) + I\left(N < i \leq 2N, j \leq \frac{N}{2}\right) \cdot \left(N^2 + g'_{A,B}\left(i - N, j, \frac{N}{2}\right)\right) \\ &+ I\left(i \leq N, \frac{3N}{2} < j \leq 2N\right) \cdot \left(N^2 + g'_{A,B}\left(i, j - N, \frac{N}{2}\right)\right) \\ &+ I\left(N < i \leq 2N, \frac{3N}{2} < j \leq 2N\right) \cdot g'_{A,B}\left(i - N, j - N, \frac{N}{2}\right) \end{aligned}$$

and

$$g'_{A,B}(i, j, 2) = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$$

4.3 Final Distribution

With Definitions 1-4 and Lemmas 1-8, we have an understanding of both the values of the reuse distances that correspond to nodes in our tree decomposition and their frequencies. We then arrive at the complete specification for a reuse distance distribution (i.e. miss ratio curve):

Theorem 2 (Reuse Distance Multiset). *The multiset of all reuse distances in an execution of $N \times N$ recursive matrix multiplication can be expressed as follows:*

$$RD_N = \underbrace{\bigcup_{l \in \{1,2,4\dots N\}} \bigcup_{(i,j,k) \in \{1..L\} \times \{1..L\} \times \{1,2\}} \{DT_{k,l}(i,j) \dots DT_{k,l}(i,j)\}}_{\frac{\#Ll}{2}} \cup \underbrace{\bigcup_{l \in \{1,2,4\dots N\}} \bigcup_{(i,j) \in \{1..L\} \times \{1..2L\}} \{F(i,j,l) \dots F(i,j,l)\}}_{\frac{N}{2l} \cdot \frac{N^2}{2l^2}} \cup \underbrace{\bigcup_{l \in \{1,2,4\dots N\}} \bigcup_{(i,j) \in \{1..2L\} \times \{1..2L\}} \{G(i,j,l) \dots G(i,j,l)\}}_{\frac{N}{2l} \cdot \frac{N^2}{4l^2}}$$

where $DT_{k,l}(i,j)$, $F(i,j,N)$, $G(i,j,N)$ and $\#Ll$ are defined in earlier lemmas and definitions.

Theorem 2 is a symbolic construction that represents RMM's reuse distance behavior for any input size as a multiset. When instantiated for a given execution, this data would normally take the form of a histogram, but as we are specifying a distribution symbolically we require this multiset construction. The elements of the set are reuse distance values, and their repetition denotes the multiplicity of that RD value.

4.4 Algorithmic form

Algorithm 1 contains a specification for how to compute a reuse distribution for RMM given the previous handful of mathematical results and an input size:

Algorithm 1 Reuse Distance Computation

Require: $RD : N \rightarrow N\{\text{Dictionary. key:RD, value:count}\}$

```

1: compute_RMM_RDD(N):
2: for  $l \in \{1, 2, 4 \dots N\}$  do
3:   for  $(i, j, k) \in \{1..L\} \times \{1..L\} \times \{1, 2\}$  do
4:     /* Lemmas 1, 2, 3, Definition 3 */
5:      $RD[DT_{k,l}(i, j)] \leftarrow RD[DT_{k,l}(i, j)] + \frac{\#Ll}{2}$ 
6:   end for
7:   for  $(i, j) \in \{1..L\} \times \{1..2L\}$  do
8:     /* Lemmas 4, 5, 6, Definition 5 */
9:      $RD[F(i, j, l)] \leftarrow RD[F(i, j, l)] + \frac{N}{2l} \cdot \frac{N^2}{2l^2}$ 
10:  end for
11:  for  $(i, j) \in \{1..2L\} \times \{1..2L\}$  do
12:    /* Lemmas 4, 7, 8, Definition 5 */
13:     $RD[G(i, j, l)] \leftarrow RD[G(i, j, l)] + \frac{N}{2l} \cdot \frac{N^2}{4l^2}$ 
14:  end for
15: end for
16: return  $RD\{\text{Dictionary stores distribution}\}$ 
```

Algorithm 1 has runtime $O(n^2 \cdot \log(n))$ for matrix dimension n , while collecting this data by running the program and performing trace analysis has runtime $O(n^3 \cdot \log(n))$ [16]. Any profiling involving running the program must be $\Omega(n^3)$,

demonstrating that our approach has guaranteed asymptotic improvement.

4.5 Verification

We verified Theorem 2 by comparing its resultant RD distribution to an RD distribution formed by instrumenting RMM and performing trace analysis. The two distributions are verified to be identical up to size 256×256 .

5 DATA MOVEMENT DISTANCE ANALYSES

In the following section we will derive bounds on the data movement distance (see Definition 1) incurred by several forms of matrix multiplication: naive, tiled, and recursive and Strassen's both with and without temporary reuse. In doing so, we demonstrate the following two valuable properties of DMD:

1. DMD is capable of asymptotically differentiating algorithms with the same time complexity as a result of their memory behavior
2. DMD is capable of asymptotically differentiating between versions of the same algorithm with and without locality optimizations

We construct precise DMD values for naive MM, asymptotically tight upper and lower bounds (differing only in coefficient) for tiled and recursive MM, and upper bounds for Strassen's algorithm and recursive MM with memory management.

5.1 Naive Matrix Multiplication

For space, we elide the derivation of naive MM's DMD. However, it is quite straightforward. We derive first is reuse distance distribution, which is very simple, then apply Definition 1 to sum the square roots of all its RDs. The result:

$$DMD_{MM} = (n^3 \cdot \sqrt{2n}) + (n^3 - 2n^2 + n) \cdot \sqrt{n^2 + 2n} + \left(\sum_{i=1}^{n-1} 2n \cdot \sqrt{n^2 + n + i} \right) + (n \cdot \sqrt{n^2 + n})$$

Simplifying asymptotically:

$$DMD_{MM} \sim n^4$$

5.2 Tiled Matrix Multiplication

Consider matrix multiplication with the computation reordered by partitioning the input matrices into $D \times D$ tiles as follows, from [2]:

```

for(jj = 0; jj < N; jj = jj + D)
  for(kk = 0; kk < N; kk = kk + D)
    for(i = 0; i < N; i = i + 1)
      for(j = jj; j < jj + D; j = j + 1)
        for(k = kk; k < kk + D; k = k + 1)
          C[i][j] = C[i][j] + A[i][k]*B[k][j]
```

For space, we elide the full derivation of tiled MM's DMD, but the approach is similar to that for naive matrix multiplication above. However, instead of deriving the precise reuse distance distribution, we form upper and lower bounded versions and use them to derive the corresponding DMD bounds. The interested reader can see the full derivation at github.com/wes-smith/Tech-Report-ICS-22/.

Theorem 3 (Tiled Matrix Multiplication DMD). *Upper and lower bounds on the data movement distance incurred by tiled matrix multiplication operating on $N \times N$ matrices with $D \times D$ tiles are as follows:*

$$\frac{N^4}{D} + N^3 \cdot D < \sim (DMD_{TMM}) < 2\sqrt{3}\frac{N^4}{D} + \sqrt{2}N^3 \cdot D$$

Note that for $D = 1$ and $D = N$, the bounds are (asymptotically) the same as the DMD of naive matrix multiplication, as tile sizes of 1 and N result in the same computation order as naive MM.

5.3 Recursive Matrix Multiplication

F :

Firstly, note that as $f_T(i, j, N) = \Omega(N^3)$ and $f_{A,B}(i, j, N) = O(N^2)$, $F(i, j, N) \sim f_T(i, j, N)$ (see Definition 5). Similarly, $G(i, j, N) \sim g_T(i, j, N)$. Noting this:

$$f_T(i, j, N) \sim T_N + 8T_{\frac{N}{2}} - \sum_{k=0}^{\log_2(N)-1} 2 \cdot T_{2^k}$$

Taking a lower bound on the summation to derive an upper bound on DMD:

$$F(i, j, N) \sim 3N^3$$

Upper bounding the summation for a lower bound:

$$F(i, j, N) \sim 2N^3$$

G has a similar analysis:

$$g_T(i, j, N) \sim 4 \cdot T_N - \sum_{k=0}^{\log_2(N)-1} 2 \cdot T_{2^k}$$

An upper bound here, bounding the summation again:

$$G(i, j, N) \sim 7 \cdot N^3$$

The corresponding lower bound:

$$G(i, j, N) \sim 6 \cdot N^3$$

5.4 Accesses to temporaries

We now turn our attention to accesses to temporaries. We will consider the four quadrants of $DT_{1,N}$, $DT_{2,N}$ separately. First, we need the asymptotic behavior of $d_1, d_2, d_3, d_4, \phi, \delta, \omega, \gamma$:

$$d_1 \sim \frac{7N^3}{2}, d_2 \sim 3N^3, d_3 \sim \frac{3N^3}{2}, d_4 \sim N^3$$

$$\phi(N), \gamma(N), \omega(N), \delta(N) \sim N^3$$

The distribution of asymptotic reuse distances for DT_1 and DT_2 are then as follows, by partitioning each into four quadrants and combining the asymptotic costs of the above:

$$DT_{1,N} : \begin{cases} 1/4 & \frac{7N^3}{2} \\ 1/4 & \frac{5N^3}{2} \\ 1/4 & 3N^3 \\ 1/4 & 2N^3 \end{cases} \quad DT_{2,N} : \begin{cases} 1/4 & \frac{3N^3}{2} \\ 1/4 & \frac{N^3}{2} \\ 1/4 & N^3 \\ 1/4 & 3N^2 \end{cases}$$

5.5 DMD calculation

With asymptotic reuse distance for each type of memory access, we can now use the frequency information in Theorem 2 to calculate DMD.

5.5.1 Temporaries. DT_1 :

$$DMD_{DT_1} = \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{2(2^i)^3}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{3(2^i)^3}}{4 \cdot 2^i}$$

$$+ \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\frac{7(2^i)^3}{2}}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\frac{5(2^i)^3}{2}}}{4 \cdot 2^i}$$

$$DMD_{DT_1} \sim \frac{2 + \sqrt{5} + \sqrt{6} + \sqrt{7} + 2\sqrt{2} + 2\sqrt{3} + \sqrt{14} + \sqrt{10}}{4} N^{3.5}$$

DT_2 :

$$DMD_{DT_2} = \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{(2^i)^3}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\frac{(2^i)^3}{2}}}{4 \cdot 2^i}$$

$$+ \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\frac{3(2^i)^3}{2}}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{3(2^i)^2}}{4 \cdot 2^i}$$

$$DMD_{DT_2} \sim \frac{3 + 2\sqrt{2} + \sqrt{3} + \sqrt{6}}{4} N^{3.5}$$

5.5.2 Accesses to A and B. First, we will consider the upper bound on F :

$$DMD_F^{up} = \sum_{i=0}^{\log_2(N)} \sum_{j=1}^{2^i} \sum_{k=1}^{2^{i+1}} \frac{N \cdot \sqrt{3(2^i)^3}}{2 \cdot 2^i} = \sum_{i=0}^{\log_2(N)} N \cdot 2^i \sqrt{3(2^i)^3}$$

$$DMD_F^{up} \sim \frac{4\sqrt{6}}{4\sqrt{2}-1} N^{3.5}$$

Next, the lower bound:

$$DMD_F^{low} = \sum_{i=0}^{\log_2(N)} \sum_{j=1}^{2^i} \sum_{k=1}^{2^{i+1}} \frac{N \cdot \sqrt{2(2^i)^3}}{2 \cdot 2^i} = \sum_{i=0}^{\log_2(N)} N \cdot 2^i \sqrt{2(2^i)^3}$$

$$DMD_F^{low} \sim \frac{8}{4\sqrt{2}-1} N^{3.5}$$

Similarly for B:

$$DMD_G^{up} = \sum_{i=0}^{\log_2(N)} \sum_{j=1}^{2^{i+1}} \sum_{k=1}^{2^i} \frac{N \cdot \sqrt{7(2^i)^3}}{2 \cdot 2^i} = \sum_{i=0}^{\log_2(N)} N \cdot 2^i \sqrt{7(2^i)^3}$$

$$DMD_G^{up} \sim \frac{4\sqrt{14}}{4\sqrt{2}-1} N^{3.5}$$

Next, the lower bound:

$$DMD_G^{low} = \sum_{i=0}^{\log_2(N)} \sum_{j=1}^{2^{i+1}} \sum_{k=1}^{2^i} \frac{N \cdot \sqrt{6(2^i)^3}}{2 \cdot 2^i} = \sum_{i=0}^{\log_2(N)} N \cdot 2^i \sqrt{6(2^i)^3}$$

$$DMD_G^{low} \sim \frac{8\sqrt{3}}{4\sqrt{2}-1} N^{3.5}$$

5.5.3 Total DMD.

Theorem 4 (Recursive Matrix Multiplication Data Movement Distance). *Combining the previous, upper and lower bounds on the data movement distance incurred by a standard recursive matrix multiplication algorithm on $N \times N$ matrices is as follows:*

$$12.82N^{3.5} < \sim (D_{RMM}(N)) < 13.46N^{3.5}$$

5.5.4 Memory Management. The RMM pseudocode analyzed earlier allocates memory on each call but contains no calls to **free()**, resulting in poor locality. Practical implementations of RMM will use memory management strategies for handling temporaries, so we will now adapt the previous DMD analysis to take this into account.

One way to introduce temporary freeing would be to call **free()** twice after each addition group, once the addition result has been stored in a C submatrix. Doing so creates the following upper bound on the total number of temporaries needed:

$$\#T(N) = N^2 + \sum_{i=0}^{\log_2(N)-1} 2 * (2^i)^2 = \frac{2}{3}(N^2 - 1)$$

$$\#T(N) < 2N^2$$

With this, a bound on the total data size of the execution (including input data) is $N^2 + N^2 + 2N^2 = 4N^2$. We previously derived reuse distances as functions of tree position: we can now reuse our analysis of the DMD of RMM without temporary reuse, but instead of using said functions, we will use the minimum of those functions and $4N^2$ (as reuse distance is always upper bounded by data size). For temporary matrix DT_1 :

$$DMD_{DT_1}^{up} = \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\min(2(2^i)^3, 4N^2)}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\min(3(2^i)^3, 4N^2)}}{4 \cdot 2^i}$$

$$+ \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\min(\frac{7(2^i)^3}{2}, 4N^2)}}{4 \cdot 2^i} + \sum_{i=0}^{\log_2(N)} \frac{N^3 \cdot \sqrt{\min(\frac{5(2^i)^3}{2}, 4N^2)}}{4 \cdot 2^i}$$

Solving for the points where the parameters to **min()** are equal, we partition each summation into two by splitting

values of induction variable i :

$$DMD_{DT_1}^{up} = \sum_{i=0}^{\lfloor \frac{\log_2(2N^2)}{3} \rfloor} \frac{N^3 \cdot \sqrt{2(2^i)^3}}{4 \cdot 2^i} + \sum_{i=\lfloor \frac{\log_2(2N^2)}{3} \rfloor+1}^{\log_2(N)} \frac{N^4}{2(2^i)}$$

$$+ \sum_{i=0}^{\lfloor \frac{\log_2(\frac{4}{3}N^2)}{3} \rfloor} \frac{N^3 \cdot \sqrt{3(2^i)^3}}{4 \cdot 2^i} + \sum_{i=\lfloor \frac{\log_2(\frac{4}{3}N^2)}{3} \rfloor+1}^{\log_2(N)} \frac{N^4}{2(2^i)}$$

$$+ \sum_{i=0}^{\lfloor \frac{\log_2(\frac{7}{2}N^2)}{3} \rfloor} \frac{N^3 \cdot \sqrt{\frac{7}{2}(2^i)^3}}{4 \cdot 2^i} + \sum_{i=\lfloor \frac{\log_2(\frac{7}{2}N^2)}{3} \rfloor+1}^{\log_2(N)} \frac{N^4}{2(2^i)}$$

$$+ \sum_{i=0}^{\lfloor \frac{\log_2(\frac{5}{2}N^2)}{3} \rfloor} \frac{N^3 \cdot \sqrt{\frac{5}{2}(2^i)^3}}{4 \cdot 2^i} + \sum_{i=\lfloor \frac{\log_2(\frac{5}{2}N^2)}{3} \rfloor+1}^{\log_2(N)} \frac{N^4}{2(2^i)}$$

Evaluating:

$$DGC_{DT_1} \sim \left(\frac{1}{2 \cdot 2^{\frac{1}{3}}} + \frac{1}{2 \cdot \frac{4}{3}^{\frac{1}{3}}} + \frac{1}{2 \cdot \frac{7}{2}^{\frac{1}{3}}} + \frac{1}{2 \cdot \frac{5}{2}^{\frac{1}{3}}} \right.$$

$$\left. + \frac{(2 + \sqrt{2})}{4} (2^{\frac{1}{6}} \cdot \sqrt{2} + \frac{4}{3} \cdot \sqrt{3} + \frac{8}{7} \cdot \sqrt{\frac{7}{2}} + \frac{8}{5} \cdot \sqrt{\frac{5}{2}}) \right) N^{\frac{10}{3}}$$

The same analysis for DT_2 results in the following:

$$DMD_{DT_2}^{up} \sim \left(\frac{1}{2 \cdot 4^{\frac{1}{3}}} + \frac{1}{2 \cdot 8^{\frac{1}{3}}} + \frac{1}{2 \cdot \frac{8}{3}^{\frac{1}{3}}} \right.$$

$$\left. + \frac{(2 + \sqrt{2})}{4} (2^{\frac{1}{3}} + 8^{\frac{1}{6}} \cdot \sqrt{\frac{1}{2}} + \frac{8}{3} \cdot \sqrt{\frac{3}{2}}) \right) N^{\frac{10}{3}}$$

We analyze F and G in the same way, but find that they are asymptotically insignificant in comparison to DT_1 and DT_2 . So, we arrive at our DMD bound for RMM with memory management:

Theorem 5 (RMM Data Movement Distance With Temporary Reuse: Upper Bound). *Combining the previous, an upper bound on the data movement distance incurred by a recursive matrix multiplication algorithm on $N \times N$ matrices employing temporary reuse is as follows:*

$$\sim (DGC_{RMM}^{up}) < 11.85N^{\frac{10}{3}}$$

5.6 Strassen's Algorithm

Strassen's algorithm for matrix multiplication, which reduces time complexity from $O(N^3)$ to around $O(N^{2.8})$ at the cost of worse locality and additional $O(N^2)$ cost, is our final target for analysis. Figure 4 [25] gives pseudocode for Strassen's, which is similar to standard RMM. The core idea is that each call to Strassen's decomposes into 7 recursive calls instead of 8 but contains additional matrix arithmetic. As before, we will analyze Strassen's both with and without locality optimizations for temporary reuse. Note in the pseudocode that, at each level of recursion, 17 temporary matrices are introduced: $M_1 \dots M_7$ store the results of recursive computation, and there are ten matrix additions or subtractions that are then passed into recursive calls. As each of these matrices is $\frac{N}{2} \times \frac{N}{2}$, the total number of temporaries needed for this call is $\frac{17N^2}{4}$. Summing over all nodes in the tree decomposition:

Algorithm	MM		RMM		Strassen	
	Naive	Tiled	Naive	Temporary Reuse	Naive	Temporary Reuse
Time Comp.	$O(N^3)$	$O(N^3)$	$O(N^3)$	$O(N^3)$	$O(N^{2.8})$	$O(N^{2.8})$
DMD	N^4	$\sqrt{2}N^3D + \frac{2\sqrt{3}N^4}{D}$	$13.46N^{3.5}$	$11.85N^{3.33}$	$6.51N^{3.4}$	$15.36N^{3.23}$

Table 1. Summary of DMD complexity compared to time complexity

$$\begin{aligned}
M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}); \\
M_2 &= (A_{21} + A_{22})B_{11}; \\
M_3 &= A_{11}(B_{12} - B_{22}); \\
M_4 &= A_{22}(B_{21} - B_{11}); \\
M_5 &= (A_{11} + A_{12})B_{22}; \\
M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}); \\
M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\
\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} &= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}
\end{aligned}$$

Figure 4. Strassen's algorithm pseudocode

Lemma 9 (Strassen's Algorithm Temporary Usage). *Let TS_N be the total number of temporaries required by an $N \times N$ execution of Strassen's algorithm where all temporaries are unique. Then*

$$TS_N = \sum_{i=1}^{\log_2(N)} \frac{17}{4} \cdot (2^i)^2 \cdot 7^{\log_2(N)-i} = \frac{17}{3} (N^{\log_2(7)} - N^2)$$

We can now asymptotically upper bound the reuse distances of each access from each quadrant of A, B as well as the temporary matrices M in terms of TS_N by counting the calls to an $\frac{N}{2} \times \frac{N}{2}$ matrix multiplication between them. For example, in Figure 4 we see that there exist three calls between the first and second uses of $A_{1,1}$, upper bounding each reuse distance from an access in $A_{1,1}$ by $3 \cdot TS_{\frac{N}{2}}$. Without enumerating all of these distances, we see there are a total of 31 such intervals, with a total distance of $96 \cdot TS_{\frac{N}{2}}$. As before, we can sum over all nodes to compute DMD:

$$DMD_{Strassen} \leq \sum_{i=1}^{\log_2(N)} \sqrt{78 \cdot TS_{2^{i-1}}} \cdot \frac{(2^i)^2}{4} \cdot 7^{\log_2(N)-i}$$

Evaluating and asymptotically simplifying:

$$\sim (DMD_{Strassen}) \leq \frac{4\sqrt{34}(N^2\sqrt{N^{\log_2(7)}} - N^{\log_2(7)})}{4\sqrt{7} - 7}$$

Theorem 6 (Strassen's Algorithm DMD: Upper Bound). *An upper bound on the data movement distance incurred by Strassen's algorithm operating on $N \times N$ matrices without temporary reuse is as follows:*

$$\sim (DMD_{Strassen}^{up}) < 6.51N^{(2+\frac{\log_2(7)}{2})}$$

$$\sim (DMD_{Strassen}^{up}) < \approx 6.51N^{3.4}$$

5.6.1 Memory Management. We will adapt the previous analysis in the same way that we did when exploring the effect of adding temporary reuse to RMM. First, we note that Huss-Lederman et al. [11] discuss a memory management technique for Strassen's that requires only N^2 temporaries. This brings total data size for an execution to $3N^2$. We will again use this data size as an upper bound on the value of an individual reuse distance.

The largest of the intervals in the pseudocode is $7 \cdot TS_{\frac{N}{2}}$, so we replace each of them with this so there is only one intersection point to consider. That shifts the sum from $96 \cdot TS_{\frac{N}{2}}$ to $31 \cdot 7 \cdot TS_{\frac{N}{2}} = 217 \cdot TS_{\frac{N}{2}}$. We must multiply our $3N^2$ upper bound by 31 as well, yielding $93N^2$.

$$DMD_{Strassen'}^{up} = \sum_{i=1}^{\log_2(N)} \sqrt{\min(217 \cdot TS_{2^{i-1}}, 93N^2)} \cdot \frac{(2^i)^2}{4} \cdot 7^{\log_2(N)-i}$$

Again we solve for an approximate equivalence point for the two functions that preserves the upper bound:

$$i \approx \log_2(0.797N^{\frac{2}{\log_2(7)}})$$

Partitioning the previous summation:

$$\begin{aligned}
DMD_{Strassen'}^{up} &= \sum_{i=1}^{\lfloor \log_2(0.797N^{\frac{2}{\log_2(7)}}) \rfloor} \sqrt{217 \cdot TS_{2^{i-1}}} \cdot \frac{(2^i)^2}{4} \cdot 7^{\log_2(N)-i} \\
&\quad + \sum_{i=\lceil \log_2(0.797N^{\frac{2}{\log_2(7)}}) \rceil}^{\log_2(N)} \sqrt{93N^2} \cdot \frac{(2^i)^2}{4} \cdot 7^{\log_2(N)-i}
\end{aligned}$$

Simplifying (while preserving the upper bound), removing asymptotically insignificant terms, and evaluating:

Theorem 7 (Strassen's Algorithm With Temporary Reuse DMD: Upper Bound). *An upper bound on the data movement distance incurred by Strassen's algorithm operating on $N \times N$ matrices with temporary reuse is as follows:*

$$\sim (DMD_{Strassen'}^{up}) < 15.36N^{3.23}$$

5.7 Summary

Table 1 contains asymptotic simplifications of the results of the previous DMD analyses, with a mix of precise results and upper bounds. We make the following observations:

- DMD is able to distinguish both between different algorithms with the same time complexity and between locality optimized vs. non-optimized versions of the same algorithm,
- The DMD reduction ($\approx N^{\frac{1}{6}}$) from adding temporary reuse is the same for RMM and Strassen even though they have different time and space complexities without temporary reuse,
- The gap between Strassen and RMM DMD is smaller than the gap between their time complexities, demonstrating that DMD has captured some of the factors that make Strassen not practical.

6 RELATED WORK

Hong and Kung [9] pioneered the study of I/O complexity, measuring memory complexity by the amount of data transfer and deriving this complexity symbolically as a function of the memory size and the problem size. The same complexity measures were used in the study of cache oblivious algorithms [8] and communication-avoiding algorithms. Olivry et al. [15] introduce a compiler technique to statically derive I/O complexity bounds. Olivry et al. use asymptotic complexity ($\sim$), much as we do, to consider constant-factor performance differences, while the rest do not. I/O complexity suffers from an issue inherited from miss ratio curves: it is not ordinal across cache sizes. Ordinality means that data can be usefully ordered; integers are ordinal, while functions are typically not. Miss ratio as a function of cache size can be numerically compared by an algorithm designer when a target cache size is known, but it cannot be used to evaluate the general effectiveness of an optimization across cache sizes. DMD, on the other hand, is an ordinal metric that is agnostic to cache size.

Memory hierarchies in practice may vary in many ways, which make a unified cost model difficult. Valiant [24] defined a bridging model, Multi-BSP, for a multi-core memory hierarchy with a set of parameters including the number of levels and the memory size and three other factors at each level. A simpler model was the uniform memory hierarchy (UMH) by Alpern et al. [1] who used a single scaling factor for the capacity and the access cost across all levels. Both are models of memory, where caching is not considered beyond the point that the memory may be so implemented.

Matrix multiplication is well researched. Much effort has been put into the analysis and optimization of the Strassen algorithm and its cache utilization [11, 17, 19, 22] as well as its parallel behavior [3, 21]. Lincoln et al. [13] explore performance in a dynamically sized caching environment.

Recently, researchers have argued that, with proper implementation considerations and under the correct conditions, the Strassen algorithm can outperform more conventionally practical variants of MM [10].

Kung and Leiserson [12] showed that matrix multiplication for $N \times N$ matrices on systolic arrays takes N steps with N^2 processors computing in parallel. At each step, data moves only between adjacent processors. Taking the wire length between neighboring processors as unit distance and ignoring the data movement outside the systolic array, the DMD is N^3 . Systolic arrays achieve the lowest possible asymptotic data movement, matching time complexity. However, the algorithm requires N^2 processors. We leave the DMD of parallel algorithms as the subject of future study.

7 CONCLUSION

We have explored the interactions between six variants of matrix multiplication and hierarchical memory through the lens of Data Movement Distance. We have demonstrated that DMD's assumptions conform with microarchitectural trends and that it is capable of exposing algorithmic properties that traditional analyses cannot. Time complexity, while an important theoretic metric by which to analyze algorithms, is at odds with a computing environment in which memory systems are increasingly large and complex. We argue that data movement complexity analysis through DMD has great potential to help engineers and algorithm designers to understand the algorithmic implications of hierarchical memory.

The results presented in this paper leave open several interesting avenues for future work. One is applying our analysis to more algorithms. Interesting targets are widely used algorithms with nontrivial memory behavior, such as convolution and Fourier transforms. Another is exploring the performance to data movement complexity relationship with empirical results. Lastly, extending this work to consider the effect of parallelism on interactions between algorithms and memory hierarchies would improve its applicability.

Acknowledgments

Donovan Snyder participated in the early stages of this project. We also thank ICS reviewers, Xiaobai Sun at Duke, Nathan Beckmann at CMU, and our colleagues at University of Rochester especially Benjamin Reber, Fangzhou Liu, John Criswell, and Michael Scott for the discussion and feedback on the DMD concept and its presentation. This work was supported in part by the National Science Foundation (Contract No. CCF-2217395, CCF-2114319, CNS-1909099). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the funding organizations.

References

- [1] B. Alpern, L. Carter, E. Feig, and T. Selker. 1994. The uniform memory hierarchy model of computation. *Algorithmica* 12, 2/3 (1994), 72–109.
- [2] Bin Bao and Chen Ding. 2013. Defensive loop tiling for shared cache. In *Proceedings of the International Symposium on Code Generation and Optimization*. 1–11.
- [3] Guy E. Blelloch, Rezaul A. Chowdhury, Phillip B. Gibbons, Vijaya Ramachandran, Shimin Chen, and Michael Kozuch. 2008. Provably Good Multicore Cache Performance for Divide-and-Conquer Algorithms (SODA '08). Society for Industrial and Applied Mathematics, USA, 501–510.
- [4] Andrew S. Cassidy and Andreas G. Andreou. 2012. Beyond Amdahl's Law: An Objective Function That Links Multiprocessor Performance Gains to Delay and Energy. *IEEE Trans. Comput.* 61, 8 (2012), 1110–1126. <https://doi.org/10.1109/TC.2011.169>
- [5] Ian Cutress. 2019. The Ice Lake Benchmark Preview: Inside Intel's 10nm. <https://www.anandtech.com/show/14664/testing-intel-ice-lake-10nm/2>
- [6] Bill Dally. [n.d.]. From Here to Exascale: Challenges and Potential Solutions.
- [7] Chen Ding and Wesley Smith. 2021. Memory Access Complexity: A Position Paper. In *Proceedings of the International Symposium on Memory Systems (MEMSYS)*.
- [8] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. 1999. Cache-Oblivious Algorithms. In *Proceedings of the Symposium on Foundations of Computer Science*. 285–298.
- [9] Jia-Wei Hong and H. T. Kung. 1981. I/O complexity: The red-blue pebble game. In *Proceedings of the ACM Conference on Theory of Computing*. Milwaukee, WI, 326–333.
- [10] Jianyu Huang, Tyler M. Smith, Greg M. Henry, and Robert A. van de Geijn. 2016. Strassen's Algorithm Reloaded. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Salt Lake City, Utah) (SC '16)*. IEEE Press, Article 59, 12 pages.
- [11] Steven Huss-Lederman, Elaine Jacobson, Jeremy Johnson, Anna Tsao, and Thomas Turnbull. 1997. Implementation of Strassen's Algorithm for Matrix Multiplication. (10 1997). <https://doi.org/10.1145/369028.369096>
- [12] H. T. Kung and Charles E. Leiserson. 1979. *Systolic Arrays for (VLSI)*. Technical Report CMU-CS-79-103. Carnegie-Mellon University.
- [13] Andrea Lincoln, Quanquan C. Liu, Jayson Lynch, and Helen Xu. 2018. Cache-Adaptive Exploration: Experimental Results and Scan-Hiding for Adaptivity. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures (Vienna, Austria) (SPAA '18)*. Association for Computing Machinery, New York, NY, USA, 213–222. <https://doi.org/10.1145/3210377.3210382>
- [14] R. L. Mattson, J. Gecsei, D. Slutz, and I. L. Traiger. 1970. Evaluation techniques for storage hierarchies. *IBM System Journal* 9, 2 (1970), 78–117.
- [15] Auguste Olivry, Julien Langou, Louis-Noël Pouchet, P. Sadayappan, and Fabrice Rastello. 2020. Automated Derivation of Parametric Data Movement Lower Bounds for Affine Programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (London, UK) (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 808–822. <https://doi.org/10.1145/3385412.3385989>
- [16] F. Olken. 1981. *Efficient methods for calculating the success function of fixed space replacement policies*. Technical Report LBL-12370. Lawrence Berkeley Laboratory.
- [17] V. Paul Pauca, Pauca Xiaobai, Sun Chatterjee, Xiaobai Sun, and Alvin Lebeck. 1998. Architecture-efficient Strassen's Matrix Multiplication: A Case Study of Divide-and-Conquer Algorithms. (07 1998).
- [18] Harald Prokop. 1999. Cache-Oblivious Algorithms.
- [19] Vikash Kumar Singh, Hemant Makwana, and Richa Gupta. 2015. Comparative Study of Cache Utilization for Matrix Multiplication Algorithms.
- [20] Donovan Snyder and Chen Ding. 2021. Measuring Cache Complexity Using Data Movement Distance (DMD). In *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 417–419.
- [21] Yuan Tang. 2020. *Balanced Partitioning of Several Cache-Oblivious Algorithms*. Association for Computing Machinery, New York, NY, USA, 575–577. <https://doi.org/10.1145/3350755.3400214>
- [22] Mithuna Thottethodi, Siddhartha Chatterjee, and Alvin Lebeck. 1998. Tuning Strassen's Matrix Multiplication for Memory Efficiency. 36–36. <https://doi.org/10.1109/SC.1998.10045>
- [23] Po-An Tsai, Nathan Beckmann, and Daniel Sanchez. 2017. Jenga: Software-Defined Cache Hierarchies (ISCA '17). Association for Computing Machinery, New York, NY, USA, 652–665. <https://doi.org/10.1145/3079856.3080214>
- [24] Leslie G. Valiant. 2008. A Bridging Model for Multi-core Computing. In *Algorithms - ESA 2008, 16th Annual European Symposium*. 13–28.
- [25] Wikipedia contributors. 2021. Strassen algorithm — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Strassen_algorithm&oldid=1049348598 [Online; accessed 24-January-2022].
- [26] Leonid Yavits, Amir Morad, and Ran Ginosar. 2014. Cache Hierarchy Optimization. *IEEE Computer Architecture Letters* 13, 2 (2014), 69–72. <https://doi.org/10.1109/L-CA.2013.18>
- [27] Liang Yuan, Chen Ding, Wesley Smith, Peter J. Denning, and Yunquan Zhang. 2019. A Relational Theory of Locality. *ACM Transactions on Architecture and Code Optimization* 16, 3 (2019), 33:1–33:26.